Project Documentation

FSD Documentation Format

Project Documentation

shopEZ E-commerce Marketplace | MERN Stack | June 2026

Phase 6: Full Stack Development (FSD) Project Documentation

Project Title: shopEZ — MERN Stack E-Commerce Marketplace
Date: June 2026
Tech Stack: MongoDB | Express.js | React.js | Node.js

---

1. Project Overview

Purpose

shopEZ is a fully functional, secure, and scalable full-stack e-commerce marketplace platform that delivers a seamless online shopping experience for customers and a powerful administrative control center for store managers and sellers.

Key Features at a Glance

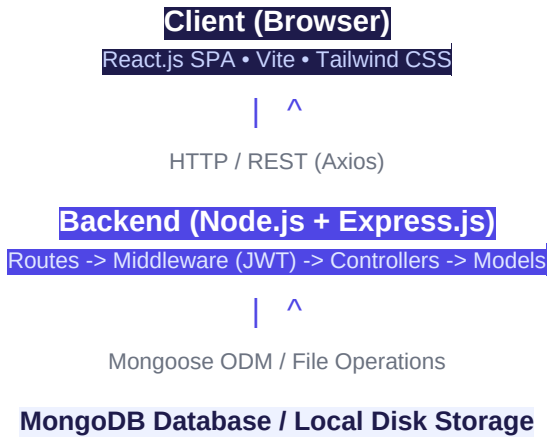
Feature Area	Highlights
Authentication	JWT-based sessions, Bcrypt password hashing, Role-Based Access Control (Customer / Seller / Admin)

Product Catalog	Live autocomplete search, multi-parameter filters (price, category, rating, stock), sort options
Shopping Cart	Persistent cross-session cart, stock validation, real-time totals
Wishlist	Save favorites, one-click "Move to Cart" with auto-clear
Checkout	Multi-step wizard (Address -> Shipping -> Coupon -> Payment), itemized billing
Payments	Mock Stripe Payment Intent endpoint mimicking production-grade integration
Reviews	Verified purchase badge, star ratings, helpful upvote tallies
Order Tracking	Visual 6-step fulfillment timeline (Placed -> Confirmed -> Packed -> Shipped -> Out for Delivery -> Delivered)
Admin Dashboard	Chart.js analytics, product/user/order/coupon CRUD, low-stock alerts

---

2. Architecture Overview

shopEZ implements the **Model-View-Controller (MVC)** pattern across a fully decoupled MERN stack:



---

3. Technology Stack

Layer	Technology	Version	Justification
Frontend Framework	React.js	19.x	Component-driven SPA with virtual DOM for blazing-fast UI updates
Build Tooling	Vite	8.x	Lightning-fast HMR development server with optimized production builds
Styling	Tailwind CSS	3.x	Utility-first responsive design with custom HSL color palettes
Routing (FE)	React Router DOM	6.x	Declarative route hierarchy with protected route guards
HTTP Client	Axios	1.7.x	Promise-based HTTP client with interceptors for auth headers
Charts	Chart.js + react-chartjs-2	4.x	Canvas-rendered bar/doughnut/line charts for admin analytics
Icons	Lucide React	0.395.x	Consistent, accessible SVG icon library
Backend Runtime	Node.js	18+	Server-side JavaScript enabling full-stack JS development
Web Framework	Express.js	4.x	Minimal, unopinionated RESTful API framework
Database	MongoDB + Mongoose	8.x	NoSQL document store with schema validation and ODM

Authentication	JWT + Bcrypt.js	—	Stateless token sessions + secure password hashing
Image Storage	Local uploads (Multer)	1.4.x	Disk-based image persistence with static file serving
Dev Fallback DB	mongodb-memory-server	11.x	In-memory MongoDB for development without external DB config
Dev Tools	Nodemon	3.x	Auto-restart server on file changes

---

4. Prerequisites

Before running shopEZ locally, ensure the following are installed:

- **Node.js** v18 or above — nodejs.org
- **npm** v8+ (bundled with Node.js)
- **MongoDB** (Local Instance OR MongoDB Atlas cloud cluster)
- **Git** for version control

---

5. Installation & Setup Guide

Step 1 — Clone the Repository

```
git clone https://github.com/<your-username>/E-Commerce-Application.git
cd "E-Commerce Application"
```

Step 2 — Configure Backend Environment Variables

Navigate to the `backend/` directory and create a `.env` file from the template:

```
cd backend
copy .env.example .env
```

Open `.env` and fill in your values:

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/shopez
JWT_SECRET=your_super_secret_jwt_key_here
```

```
STRIPE_API_KEY=your_stripe_api_key_here
```

> **Note:** If `MONGO_URI` is unreachable, the seeder automatically starts a local `mongodb-memory-server` in-memory fallback and seeds it with sample customers, products, reviews, and active coupons.

Step 3 — Install Backend Dependencies

```
# Inside /backend
npm install
```

Step 4 — Install Frontend Dependencies

```
cd ../frontend
npm install
```

Step 5 — (Optional) Seed the Database

```
cd ../backend
npm run seed
```

Seeds the database with mock: admin account, customer accounts, 20+ products across multiple categories, active promotional coupon codes, and sample reviews.

---

6. Running the Application

Development Mode (Full Stack)

Terminal 1 — Backend API Server:

```
cd backend
npm run dev
# Server starts on http://localhost:5000
```

Terminal 2 — Frontend Dev Server:

```
cd frontend
npm run dev
# App starts on http://localhost:5173
```

---

7. Folder Structure

Backend (`/backend`)

```
backend/
|-- config/
|   `-- db.js                 # MongoDB connection (with in-memory fallback)
|-- controllers/
|   |-- authController.js     # User register, login, profile, wishlist, addresses
|   |-- productController.js# Catalog search, reviews, helpful votes
|   |-- orderController.js    # Order creation, payment update, delivery milestones
|   |-- couponController.js # Coupon CRUD, active validation
|   `-- analyticsRoutes.js  # Sales charts & revenue aggregation
|-- middleware/
|   |-- authMiddleware.js     # JWT protect(), admin(), seller() guards
|   `-- errorMiddleware.js  # Centralized notFound & errorHandler
|-- models/
|   |-- User.js               # User schema (addresses[], wishlist[], role)
|   |-- Product.js            # Product schema (reviews[], stock, rating)
|   |-- Order.js              # Order schema (items, payment, delivery timeline)
|   |-- Review.js             # Review schema (verified, helpfulVotes)
|   |-- Coupon.js             # Coupon schema (code, type, value, expiry)
|   `-- Seller.js             # Seller profile schema
|-- routes/
|   |-- authRoutes.js         # /api/users (register, login, profile, addresses)
|   |-- productRoutes.js      # /api/products (list, CRUD, reviews, suggestions)
|   |-- orderRoutes.js        # /api/orders (create, pay, deliver)
|   |-- couponRoutes.js       # /api/coupons (admin CRUD, user validate)
|   |-- uploadRoutes.js       # /api/upload (Multer file upload)
|   `-- analyticsRoutes.js  # /api/analytics (sales metrics)
|-- utils/
|   `-- seeder.js             # Database seed script
|-- uploads/                  # Local product image storage
|-- server.js                 # Express app entry point
`-- .env.example              # Environment variable template
```

Frontend (`/frontend`)

```
frontend/
|-- src/
|   |-- assets/               # Static assets (logos, icons)
|   |-- components/
|   |   |-- Navbar.jsx        # Top navigation with cart badge, auth links
|   |   |-- ProductCard.jsx # Reusable product listing card
|   |   `-- ProtectedRoute.jsx  # Route guard (auth + role verification)
|   |-- context/
|   |   |-- AuthContext.jsx # User session state & dispatch
|   |   |-- CartContext.jsx # Shopping cart state (items, totals)
|   |   `-- ToastContext.jsx# Global toast notification system
|   |-- pages/
|   |   |-- Home.jsx            # Catalog browse with search & filters
|   |   |-- ProductDetails.jsx  # Single product, reviews, related carousel
|   |   |-- Cart.jsx            # Cart summary with quantity controls
|   |   |-- Wishlist.jsx        # Saved wishlist with move-to-cart
|   |   |-- Checkout.jsx        # Multi-step wizard with coupon & mock payment
|   |   |-- MyOrders.jsx        # Order history with fulfillment tracker
|   |   |-- ProfileDashboard.jsx# Edit profile, manage addresses
|   |   |-- AdminDashboard.jsx  # Admin panel (analytics, CRUD, coupons)
```

```
|   |   |-- SellerDashboard.jsx # Seller panel (products, orders)
|   |   |-- Login.jsx           # Sign-in form
|   |   `-- Register.jsx        # Registration form
|   |-- utils/                  # Helper functions and Axios config
|   |-- App.jsx                 # Root routes definition
|   `-- main.jsx                # ReactDOM entry point
|-- index.html                  # Vite HTML template
`-- vite.config.js              # Vite configuration
```

---

8. RESTful API Documentation

All API routes are served from `http://localhost:5000`. Protected routes require a valid `Authorization: Bearer <jwt_token>` header.

8.1 Authentication Routes — `/api/users`

Method	Endpoint	Auth	Description
`POST`	`/api/users`	Public	Register a new user account
`POST`	`/api/users/login`	Public	Log in and retrieve JWT token
`GET`	`/api/users/profile`	User	Get current user profile
`PUT`	`/api/users/profile`	User	Update name, email, or password
`GET`	`/api/users/wishlist`	User	Retrieve user's wishlist products
`POST`	`/api/users/wishlist/:productId`	User	Toggle product in/out of wishlist
`POST`	`/api/users/addresses`	User	Add a new shipping address
`PUT`	`/api/users/addresses/:addressId`	User	Edit an existing shipping address
`DELETE`	`/api/users/addresses/:addressId`	User	Remove a shipping address

`GET`	`/api/users`	Admin	Get all registered users
`PUT`	`/api/users/:id/block`	Admin	Block or unblock a user account

8.2 Product Routes — `/api/products`

Method	Endpoint	Auth	Description
`GET`	`/api/products`	Public	List all products with optional search/filter/sort
`GET`	`/api/products/suggestions`	Public	Get autocomplete name suggestions
`GET`	`/api/products/:id`	Public	Get single product detail
`GET`	`/api/products/:id/related`	Public	Get related product recommendations
`POST`	`/api/products`	Seller/Admin	Create a new product listing
`PUT`	`/api/products/:id`	Seller/Admin	Update product information
`DELETE`	`/api/products/:id`	Seller/Admin	Delete a product listing
`POST`	`/api/products/:id/reviews`	User	Submit a product review with star rating
`POST`	`/api/products/:id/reviews/:reviewId/vote`	User	Toggle helpful vote on a review

8.3 Order Routes — `/api/orders`

Method	Endpoint	Auth	Description

`POST`	`/api/orders`	User	Place a new order (deducts stock)
`GET`	`/api/orders/myorders`	User	Retrieve current user's order history
`GET`	`/api/orders/:id`	User	Get single order details
`PUT`	`/api/orders/:id/pay`	User	Update order payment status to paid
`GET`	`/api/orders`	Admin	List all platform orders
`PUT`	`/api/orders/:id/deliver`	Admin	Update order delivery status milestone

8.4 Coupon Routes — `/api/coupons`

Method	Endpoint	Auth	Description
`GET`	`/api/coupons`	Admin	List all coupon codes
`POST`	`/api/coupons`	Admin	Create a new coupon rule
`PUT`	`/api/coupons/:id`	Admin	Edit coupon details or toggle active status
`DELETE`	`/api/coupons/:id`	Admin	Delete a coupon permanently
`POST`	`/api/coupons/validate`	User	Validate coupon code and retrieve discount

8.5 Payment Route — `/api/payment`

Method	Endpoint	Auth	Description
`POST`	`/api/payment/create-intent`	User	Create a mock Stripe payment intent returning a client secret

8.6 Upload Route — `/api/upload`

Method	Endpoint	Auth	Description
`POST`	`/api/upload`	Seller/Admin	Upload a product image via Multer (stored in `/uploads`)

---

9. Authentication & Security Model

- **Password Hashing:** All user passwords are hashed via `bcryptjs` with a salt round factor of 10 before being persisted to MongoDB.
- **Session Tokens:** Upon successful login, the server signs a JWT containing `user._id` and `user.role` with a configurable secret (`JWT_SECRET`). Tokens carry a 30-day expiry.
- **Protected Routes (Backend):** The `protect` middleware in authMiddleware.js verifies the `Authorization` header on every protected endpoint. The `admin()` and `seller()` middleware further restrict access based on the decoded role field.
- **Protected Routes (Frontend):** The ProtectedRoute.jsx component wraps sensitive React pages, redirecting unauthenticated users to the Login page.

---

10. Known Constraints

- **Payment Integration:** The current Stripe integration uses a simulated mock endpoint. A real Stripe publishable key and a configured webhook listener would be required for live transaction processing.
- **Image Storage:** Product images are stored locally in the `backend/uploads/` directory. For production deployments, replacing Multer's disk storage with Cloudinary or AWS S3 integration is recommended.
- **Seller Profile:** Seller onboarding requires admin approval via the admin panel before a seller account gains product management privileges.

---

11. Future Enhancements

- **Real Payment Gateway Integration:** Wire actual Stripe webhook handling to confirm payment events asynchronously.
- **Cloud Image CDN:** Integrate Cloudinary SDK to replace local Multer disk storage.
- **AI Product Recommendations:** Implement ML-driven collaborative filtering to personalize related-product suggestions.
- **Multi-Vendor Marketplace:** Expand seller onboarding to support independent vendor profiles, individual seller pages, and commission tracking.
- **Advanced Analytics:** Add time-series graphs, cohort retention charts, and inventory forecasting to the Admin Dashboard.

- **Push Notifications:** Notify customers via email or browser push on order status milestones (Shipped, Delivered).